The String Pool and intern()

9 min read Updated 28 Jul 2026 Topic 13 of 65

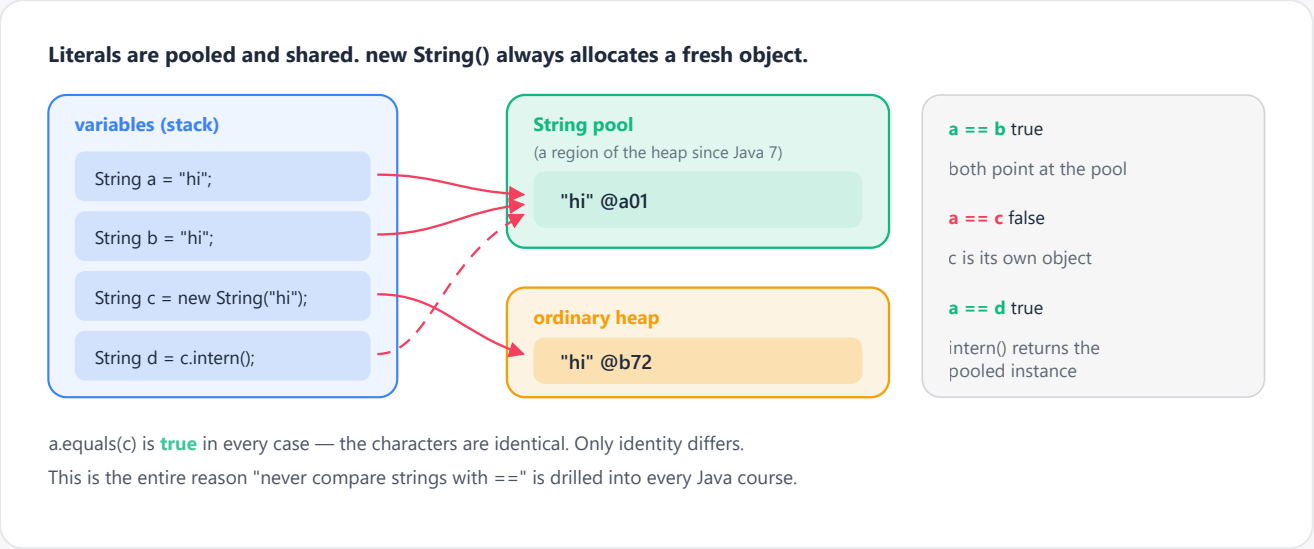
Bookmark Mark done

Why it matters

The string pool is the reason `==` sometimes appears to work on strings — which makes it the single most effective trap in Java interviews. A candidate who has only ever compared literals may believe `==` is fine, and the code works right up until a string arrives from a database or a request body. Understanding the pool turns a memorised rule ("always use `equals`") into knowledge you can reason from.

The concept, simply

The **string pool** (or string constant pool) is a table of unique `String` objects the JVM maintains. Every string *literal* in your source is automatically placed there, and every occurrence of the same literal anywhere in the program refers to that one object. `new String("hi")` deliberately bypasses the pool and allocates a fresh object.



Two literals share one pooled object, so == is true. new String() allocates separately, so == is false — even though the characters are identical.

How it works

Literals are pooled at class load time: the constant pool of the `.class` file contains the text, and on first use the JVM either finds an existing pooled `String` with those characters or creates one and adds it. Every later use of that same literal — in any class, anywhere in the program — resolves to the identical object.

Compile-time constant folding is the part that surprises people. If the compiler can evaluate a concatenation entirely at compile time, the result is a literal and therefore pooled:

JAVA Copy

```
String a = "hello";
String b = "hel" + "lo";              // folded by javac into the literal
System.out.println(a == b);           // true

final String part = "hel";            // final and initialised with a cons
String c = part + "lo";               // still a compile-time constant
System.out.println(a == c);           // true

String prefix = "hel";                // NOT final → not a compile-time co
String d = prefix + "lo";             // built at RUN time, on the heap
System.out.println(a == d);           // false
System.out.println(a.equals(d));      // true  ← the only comparison that
```

That is the whole trap in four lines: whether `==` is true depends on whether `javac` could fold the expression, which depends on a `final` keyword somewhere else entirely. It is a superb illustration of why `==` on strings is never a design you should rely on.

`intern()` is the manual entry point: `s.intern()` returns the pooled instance with the same characters, adding `s` to the pool if no such entry exists. It is occasionally useful for deduplicating a very large number of repeated strings read from input, but it is rarely the right tool — it is not free, and the JVM has a better mechanism (see below).

Code examples

The four comparisons every interviewer draws from:

JAVA Copy

```java
String a = "hi";
String b = "hi";
String c = new String("hi");
String d = c.intern();

a == b        // true  — both are the pooled literal
a == c        // false — c is a distinct heap object
a == d        // true  — intern() returned the pooled instance
c == d        // false — c is still its own object
a.equals(c)   // true  — the ONLY comparison that reflects meaning
```

Where this becomes a real production bug:

JAVA Copy

```java
// Works in the unit test, fails against the database.
String status = resultSet.getString("status");   // built at run time
if (status == "ACTIVE") {                        // ✗ always false
    // never reached
}
if ("ACTIVE".equals(status)) {                   // ✓ and null-safe, in th
    // correct
}
// Note the ordering: literal.equals(variable) cannot throw if the variabl
// is null. Or use Objects.equals(a, b), which is null-safe either way.
```

`switch` on strings, for completeness, does the right thing automatically:

JAVA Copy

```java
switch (status) {                    // uses hashCode() then equals() interna
    case "ACTIVE"   -> activate();
    case "INACTIVE" -> deactivate();
    default         -> log(status);
}
```

Under the hood

The pool moved, and it matters. Before Java 7 the string pool lived in PermGen, which was small and fixed — so interning many strings caused `OutOfMemoryError: PermGen space`. In Java 7 it moved to the main heap, which means pooled strings are now ordinary garbage-collectible objects subject to the normal heap limits. It is implemented as a fixed-bucket hash table whose size is tunable with `-XX:StringTableSize`; a program that interns millions of strings can degrade badly if that table is too small for the load.

The modern alternative to `intern()` is worth knowing, because it is the answer to "how would you reduce memory from duplicate strings?". The G1 collector offers **string deduplication** (`-XX:+UseStringDeduplication`), which detects `String` objects with identical contents during GC and makes them share one backing byte array. It gets most of the memory benefit of interning with none of the application-code changes and none of the pool-contention risk. Naming it distinguishes someone who has tuned a real JVM from someone who has read about `intern()`.

One last mechanism: at run time, `+` on non-constant strings does not literally allocate intermediate strings the way older answers claim. Since Java 9, `javac` compiles string concatenation to an `invokedynamic` call, and the JDK's `StringConcatFactory` generates an optimised method at first execution — typically sizing the result exactly and filling it in one pass. This makes a single concatenation expression genuinely efficient. It does *not* rescue concatenation in a *loop*, because each iteration is a separate expression that must produce a complete new string.

Common mistakes

- **Comparing strings with `==`.** It works for literals and fails for anything computed, parsed or read from I/O — the worst possible failure mode.
- **Using `new String("literal")`.** It has no purpose except to defeat pooling. Always write the literal.

- **Calling `intern()` defensively.** It adds pool pressure and lock contention for a benefit you probably do not need. Prefer G1 string deduplication.
- **Assuming the pool is in PermGen.** It moved to the heap in Java 7.
- **Writing `variable.equals("LITERAL")`.** Reverse it — `"LITERAL".equals(variable)` — and the NPE disappears.
- **Believing every `+` creates garbage.** A single expression is optimised via `invokedynamic`; it is the loop that is the problem.

Interview lens

Q: "What is the string pool?"
A: A JVM-maintained table of unique `String` objects. Every literal is placed there automatically, so all occurrences of the same literal across the whole program share one object. It exists because strings are immutable, which makes sharing safe, and it saves a great deal of memory given how many duplicate literals a typical program contains.

Q: "`String a = "hi"; String b = new String("hi");` — is `a == b`?"
A: No. `a` refers to the pooled literal; `new String` explicitly allocates a separate object. `a.equals(b)` is true because the characters match. `b.intern()` would return the pooled instance, and that *would* be `==` to `a`.

Q: "Why is `"hel" + "lo" == "hello"` true, but not when a variable is involved?"
A: Constant folding. If `javac` can evaluate the expression at compile time — both operands are literals, or a `final` variable initialised with a constant — the result becomes a literal and is pooled, so `==` holds. With a non-final variable the concatenation happens at run time and produces a new heap object. That the answer depends on a `final` keyword elsewhere is the best argument there is for never using `==` on strings.

A: It returns the pooled instance for those characters, adding the string to the pool if absent. Legitimately useful when a program holds an enormous number of duplicate strings from input and memory matters. But it costs pool insertion and contention, so I would normally reach for G1's `-XX:+UseStringDeduplication` instead — the GC merges identical strings' backing arrays with no code changes.

Q: "Where does the pool live?"
A: On the heap since Java 7. Before that it was in PermGen, which was fixed-size and could be exhausted by heavy interning. Moving it to the heap made pooled strings ordinary collectible objects. The table itself is a fixed-bucket hash table sized by `-XX:StringTableSize`.

Q: "How does `switch` on a String work?"
A: The compiler generates a two-step lookup: switch on `hashCode()` to narrow the candidates, then `equals()` to confirm — so it is correct regardless of pooling. It throws `NullPointerException` on a null selector in classic switch; Java 21's pattern-matching switch lets you write `case null`.

Cheat-card

- Literals are **pooled automatically**; `new String( ... )` deliberately is not.
- `==` compares identity → true for literals, **false for anything computed at run time**.
- **Constant folding**: `"hel" + "lo"` is a literal; with a non-final variable it is not.
- `intern()` returns the pooled instance. Rarely the right tool.
- Pool moved **PermGen → heap in Java 7**. Table size: `-XX:StringTableSize`.
- Better memory fix: **G1** `-XX:+UseStringDeduplication`.
- Always `"LITERAL".equals(var)` or `Objects.equals(a, b)` — null-safe.
- Java 9+: a single `+` expression compiles to `invokedynamic` and is efficient.

1. String a = "hi"; String b = new String("hi"); What are a == b and a.equals(b)?

- ◯ true and false
- ◯ false and true — b is a distinct heap object, but the characters match
- ◯ true and true
- ◯ false and false

Check answer

2. Why is "hel" + "lo" == "hello" true, but prefix + "lo" == "hello" false when prefix is a non-final variable holding "hel"?

- ◯ It is a JVM bug fixed in Java 9
- ◯ The + operator behaves differently on variables
- ◯ Constant folding: javac evaluates the all-literal expression at compile time so it becomes a pooled literal, while the variable version is built on the heap at run time
- ◯ Non-final variables are never pooled at all, including their contents

Check answer

3. What is the modern alternative to calling intern() to reduce memory from duplicate strings?

- ◯ Increasing -XX:StringTableSize
- ◯ Calling System.gc() more often
- ◯ Storing strings as char[] instead

G1 string deduplication (-XX:+UseStringDeduplication), which merges identical strings' backing arrays during GC with no code changes

Check answer

Previous
Why String Is Immutable

Next
String vs StringBuilder vs StringBuffer

LEARN MORE

ARTICLES & DOCS

String.intern() — API docs
docs.oracle.com

String literals (JLS §3.10.5)
docs.oracle.com

VIDEOS



String Pool | String Buffer | String Builder | Java Interview Questions And Answers
Code Crush



Java Strings Complete Guide | String Pool, Immutability, StringBuilder, StringBuffer & Differences
codemonarch

Hand-picked references for this lesson.